

FitBuddy Project Report

AI-Powered Personal Fitness & Nutrition Planning Ecosystem

TEAM LEADER: M. MAHALAKSHMI
Author: Dhanushka & Tamizh

Architecture: FastAPI & Google Gemini API (Flash)

Version: 1.0.0 (Production)

Date: October 2026

1. Executive Summary

FitBuddy is a modern, responsive web application engineered to solve the common disconnect between generalized fitness advice and individual body mechanics. By leveraging cutting-edge Large Language Model capabilities through the Google Gemini API, FitBuddy evaluates parameters such as body weight, age, specific fitness goals (e.g., weight loss, muscle gain), and training intensity to instantly compute highly customized 7-day workout routines and targeted nutritional guidance.

> **Core Objective:** Democratize elite-tier personal coaching through lightweight, self-hosted local deployments powered by asynchronous backend architectures.

Key Highlights

- **Dynamic Prompt Generation:** Translates rigid user configurations into context-rich, instructional prompts for AI generation.
- **Dual-Engine Processing:** Separates core workout calculation from nutrition/recovery advice using concurrent modular execution.
- **Low Latency Response:** Optimized via Google's latest `models/gemini-3.8-flash` pipeline for high-speed output rendering.

This report documents the architectural design, codebase layout, API integration challenges, configuration workflows, version control protocols, and deployment strategies for the complete FitBuddy software suite.

2. System Architecture

The FitBuddy architecture follows a streamlined Client-Server model. The client layer consists of a clean HTML5/CSS3 presentation frontend running directly in standard web browsers, communicating via HTTP requests with an asynchronous backend built on FastAPI.

Architectural Flow Diagram

```
      [ Web Browser / UI ] │ (HTTP POST / Form Submission) ▼ [ FastAPI Backend
Router (main.py / routes.py) ] ├──► [ Gemini Generator Module ] ──► (Google
Gemini API - Workout Plan) └──► [ Flash Generator Module ] ──► (Google Gemini API
- Nutrition Tip) │ ▼ [ JSON / HTML Structured Response Rendered to User ]
```

Design Principles

- **Asynchronous Non-Blocking IO:** FastAPI's native async capabilities ensure optimal resource utilization during high-concurrency API calls.
- **Modular Separation of Concerns:** Routing logic, AI integrations, and configuration parameters are cleanly decoupled into isolated modules.

3. Backend Core Components

The backend is powered by Python 3.9+ and uses Uvicorn as an ASGI server framework. The core entry point coordinates routing and middleware validation.

Application Initialization (`main.py`)

```
      from fastapi import FastAPI from fastapi.staticfiles import StaticFiles from app.routes import router app = FastAPI(title="FitBuddy API", version="1.0") # Mount static assets and router app.mount("/static", StaticFiles(directory="static"), name="static") app.include_router(router)
```

The application instance initializes middleware handlers, sets up CORS compliance if required, and maps endpoint routes cleanly for downstream consumers.

4. Gemini Integration & Workout Generator

The workout generator module (`gemini_generator.py`) handles complex prompt construction, embedding personal metrics to prompt the AI for structured 7-day programs.

```
      import os import google.generativeai as genai from dotenv import load_dotenv load_dotenv() genai.configure(api_key=os.getenv("GEMINI_API_KEY")) def generate_workout_gemini(name: str, age: int, weight: float, goal: str, intensity: str, feedback: str = None): model = genai.GenerativeModel('models/gemini-3.8-flash') prompt = f"Create a personalized 7-day workout plan for {name}, Age: {age}, Weight: {weight}kg, Goal: {goal}, Intensity: {intensity}." try: response = model.generate_content(prompt) return response.text except Exception as e: return f"Error generating workout plan: {str(e)}"
```

This module enforces model version consistency, pointing directly to `models/gemini-3.8-flash` to avert legacy 404 lookup failures.

5. Flash Generator & Nutrition Engine

Complementing the main workout generator, the flash generator (`gemini_flash_generator.py`) isolates lightweight inference calls designed to deliver snappy dietary and recovery advice.

```
      import os import google.generativeai as genai from dotenv import load_dotenv
load_dotenv() genai.configure(api_key=os.getenv("GEMINI_API_KEY")) def
generate_nutrition_tip_with_flash(goal: str) -> str: model =
genai.GenerativeModel("models/gemini-3.8-flash") prompt = f"Provide a concise,
practical nutrition or recovery tip tailored for goal: '{goal}'." try: response =
model.generate_content(prompt) return response.text except Exception as e: return
f"Nutrition tip unavailable: {str(e)}"
```

By decoupling nutrition retrieval from heavy multi-day workout processing, the application maintains high responsiveness.

6. Dependency Management & Environment

FitBuddy's environment isolation relies on Python virtual environments (`venv`) and pinned package requirements to guarantee cross-system reproducibility.

Requirements Specification (`requirements.txt`)

```
      fastapi==0.110.0 uvicorn==0.28.0 google-generativeai==0.4.1 python-
dotenv==1.0.1 pydantic==2.6.4
```

Virtual Environment Setup Steps

- Create isolated environment: `python -m venv fitbuddy-env`
- Activate environment (Windows PowerShell/CMD): `fitbuddy-env\Scripts\activate`
- Install dependencies: `pip install -r requirements.txt`

7. API Error Handling & Version Evolution

During the development lifecycle, API version migrations required robust troubleshooting to address legacy model deprecations and endpoint naming changes.

Error Encountered	Root Cause	Resolution Implemented
404 Model Not Found	Deprecated endpoint naming (`gemini-1.5-flash`)	Migrated to `models/gemini-3.8-flash`
AttributeError: response.texts	Typo in response accessor property	Corrected to standard `response.text`
Command Not Found: git	Git CLI missing from system PATH	Installed Git for Windows & added to PATH

Proper exception wrappers prevent application crashes when upstream API limits or network timeouts occur.

8. Frontend User Interface

The user interface provides a clean, ergonomic form submission mechanism enabling users to specify their biometric parameters with zero friction.

Form Parameters Captured

- **User Name:** Personalized string identifier for custom greetings.
- **Age & Weight:** Numerical inputs used to scale exercise intensity.
- **Fitness Goal:** Dropdown selector (Weight Loss, Muscle Gain, Endurance, General Fitness).
- **Intensity Preference:** Low, Medium, or High workout pacing.

Dynamic JavaScript fetch calls transmit payload data asynchronously to FastAPI endpoints, rendering formatted Markdown output directly within the browser dashboard.

9. Version Control & Git Management

Managing source code history and collaboration requires disciplined Git workflows. FitBuddy tracks all code iterations locally before synchronizing with remote GitHub repositories.

Standard Git Operational Sequence

```
      git init git add . git commit -m "initial commit: modularized backend &
Gemini integration" git remote add origin https://github.com/username/fitbuddy.git
git push origin main
```

For Windows environments lacking pre-configured terminal Git binaries, alternative manual browser-based uploading workflows via GitHub's web interface provide a reliable fallback.

10. Security & Configuration Management

Protecting sensitive API credentials is paramount. FitBuddy isolates all secrets using environment variables managed by `python-dotenv`.

Environment Configuration (`.env`)

```
GEMINI_API_KEY=AIzaSyD-YourActualAPIKeyHere-Placeholder
```

Security Best Practices

- **Exclusion Rules:** The `.env` file is explicitly included in `.gitignore` to prevent accidental credential leaks to public version control repositories.
- **Key Rotation:** Instructions are provided for rotating compromised keys immediately through Google AI Studio dashboards.

11. Testing & Server Verification

Local verification ensures that the FastAPI server reloads cleanly and processes model requests without latency spikes.

Uvicorn Startup Log Verification

```
      INFO: Started server process [29652] INFO: Waiting for application startup.
INFO: Application startup complete. INFO: Uvicorn running on http://127.0.0.1:8000
(Press CTRL+C to quit)
```

Live endpoint testing confirms successful JSON payload transmission and Markdown rendering for custom user profiles.

12. Performance Optimization

Optimizing throughput involves minimizing blocking operations and streamlining model prompt token counts.

- **Token Economy:** Prompts are structured concisely to reduce token consumption and accelerate inference times on Google's flash models.
- **Hot Reloading:** Development servers utilize Uvicorn's stat reload feature to reflect code modifications instantly without manual server restarts.
- **Static Asset Caching:** CSS stylesheets and client scripts are served with optimized caching headers.

13. Deployment & Production Guide

Moving FitBuddy from local development to cloud production environments requires specific hosting configurations.

Production Hosting Recommendations

- **Process Managers:** Use `systemd` or Docker containers to manage Uvicorn worker processes reliably in Linux server environments.
- **Reverse Proxy:** Deploy Nginx in front of Uvicorn to handle SSL termination and static file serving.
- **Environment Injection:** Inject production API keys securely via cloud provider secret managers rather than flat `.env` files.

14. Future Roadmap & Enhancements

The FitBuddy roadmap outlines several advanced features planned for subsequent major releases:

- **Database Persistence:** Integrating SQLite or PostgreSQL to save user workout histories and track longitudinal weight loss progress.
- **User Authentication:** Implementing OAuth2 JWT login tokens for multi-user account isolation.
- **Mobile Companion App:** Developing a React Native cross-platform mobile interface communicating with the FastAPI backend.
- **Wearable Sync:** Integrating Apple Health and Google Fit APIs for real-time caloric expenditure feedback.

15. Complete Project Directory Layout

The hierarchical tree structure of the repository ensures clean separation of concerns across backend logic, environment assets, and documentation:

```
     fitbuddy/ ├── backend/ | ├── app/ | | ├── __init__.py | | ├── main.py | | ├── routes.py | | ├── gemini_generator.py | | └── gemini_flash_generator.py | ├── fitbuddy-env/ | | ├── Include/ | | ├── Lib/ | | └── Scripts/ | └── requirements.txt ├── .env ├── .gitignore └── README.md
```

16. Appendix A: Complete Routing Logic

The routing module coordinates incoming web requests, invokes the respective AI generator functions, and aggregates responses for the presentation layer.

```
      from fastapi import APIRouter, Form from app.gemini_generator import
generate_workout_gemini from app.gemini_flash_generator import
generate_nutrition_tip_with_flash router = APIRouter() @router.post("/generate")
async def handle_generation(name: str = Form(...), age: int = Form(...), weight:
float = Form(...), goal: str = Form(...), intensity: str = Form(...)): workout =
generate_workout_gemini(name, age, weight, goal, intensity) nutrition =
generate_nutrition_tip_with_flash(goal) return {"name": name, "workout": workout,
"nutrition": nutrition}
```

17. Appendix B: Quick Troubleshooting Reference

A handy operational reference guide for common developer hurdles encountered during local deployment:

- **Port 8000 Already in Use:** Kill active Python processes or run Uvicorn on an alternate port using `--port 8001`.
- **ModuleNotFoundError:** Ensure the virtual environment is activated (`fitbuddy-env\Scripts\activate`) before executing scripts.
- **API Quota Exceeded:** Verify Google AI Studio billing tiers and rate-limit thresholds for flash model endpoints.
- **Git Authentication Failures:** Utilize personal access tokens (PAT) or SSH keys when pushing code updates to GitHub repositories.

18. Conclusion & Sign-Off

FitBuddy successfully demonstrates the practical integration of generative AI within lightweight web frameworks. By overcoming initial API version hurdles and establishing robust local version control workflows, the project stands ready for continuous expansion and deployment.

> **Project Status:** Successfully Deployed Locally & Documented
>
> **Approved By:** Engineering Team (Dhanushka & Tamizh)
>
> **Repository:** GitHub Main Branch Synchronized

End of Comprehensive FitBuddy Report — 2026